Question 1)

Option 4 is incorrect.

Question 2)

Option 3 is incorrect.

Question 3)

The EM algorithm is considered a greedy algorithm because at each iteration it makes a locally optimal decision by maximizing the likelihood function based on the current parameter estimates. In the E-step, it estimates the hidden variables using existing parameters, and in the M-step, it updates the parameters to maximize the likelihood given those estimates. This step-by-step process always improves or maintains the likelihood but does not consider the global solution space, meaning it can converge to a local maximum rather than the global optimum. Hence, its local, iterative maximization behavior is analogous to greedy optimization.

Question 4)

Applications of Clustering Algorithms:

Data Analysis – Grouping similar data points to discover patterns and trends.

Customer Segmentation – Dividing customers into groups based on behavior or purchasing habits.

Market Research – Identifying target customer groups for products/services.

Image Segmentation – Dividing images into meaningful regions in computer vision.

Anomaly / Fraud Detection – Detecting unusual patterns in banking or cybersecurity.

Document / Text Clustering – Grouping similar documents or news articles.

Gene Expression Analysis – Grouping genes with similar biological behavior.

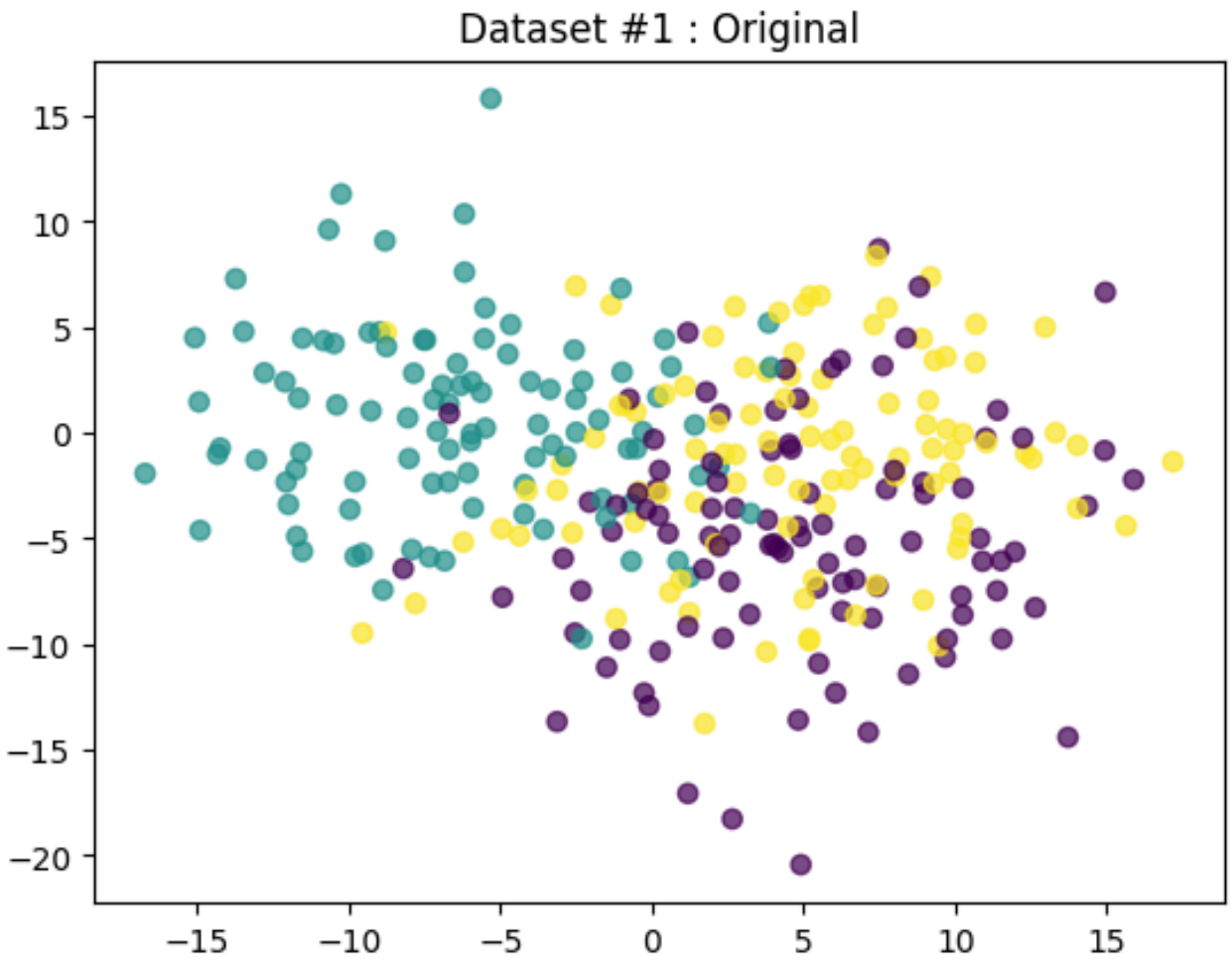
Question 5)

1. Elbow method:The Elbow Method selects K by analyzing how the clustering error changes as K increases. First, run K-Means for different values of K (for example, K = 1 to 10). For each K, compute the Within-Cluster Sum of Squares (WCSS), which measures the total distance between data points and their respective cluster centroids. Then plot K on the x-axis and WCSS on the y-axis. As K increases, WCSS decreases because clusters become tighter. However, after a certain point, the rate of decrease sharply slows down, forming an “elbow” shape in the graph. The value of K at this elbow point is considered the optimal number of clusters because adding more clusters beyond this gives only marginal improvement.
2. Silhouette Method:The Silhouette Method evaluates how well each data point fits within its assigned cluster compared to other clusters. For each K, compute the Silhouette Score, which ranges from –1 to +1. A higher score means data points are well matched to their own cluster and poorly matched to neighboring clusters. To find the optimal K, run K-Means for multiple K values and calculate the average silhouette score for each. Plot K versus the silhouette score, and choose the K with the highest silhouette value, as it indicates the best cluster separation and cohesion.

Question 6)

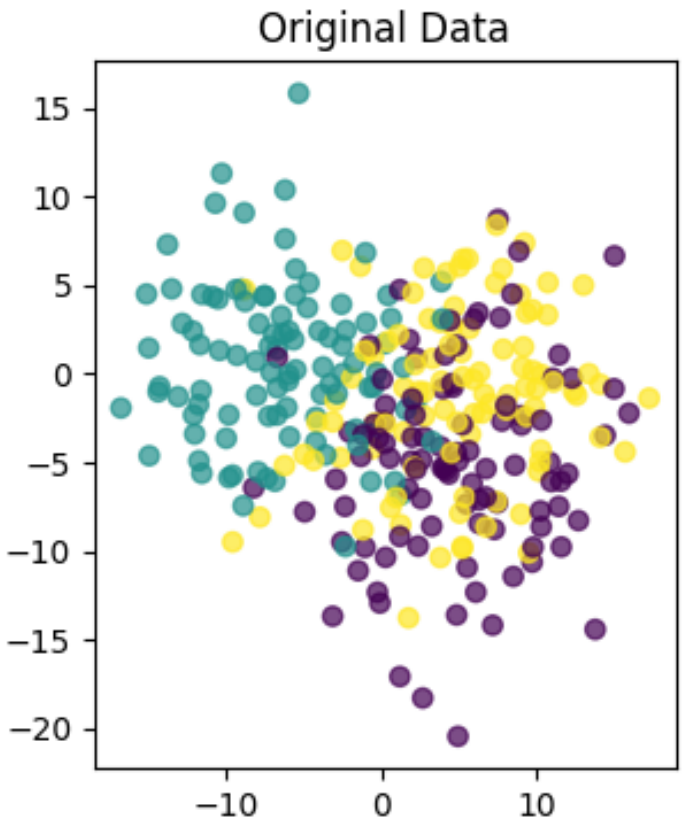
```
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans
```

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.cluster import DBSCAN
import pandas as pd
import matplotlib.pyplot as plt
X, label = make_blobs(n_samples=300, n_features=2, centers=3, cluster_std=5, random_state=123)
plt.scatter(X[:,0], X[:,1], c=label, alpha=0.7)
plt.title('Dataset #1 : Original')
plt.show()
```



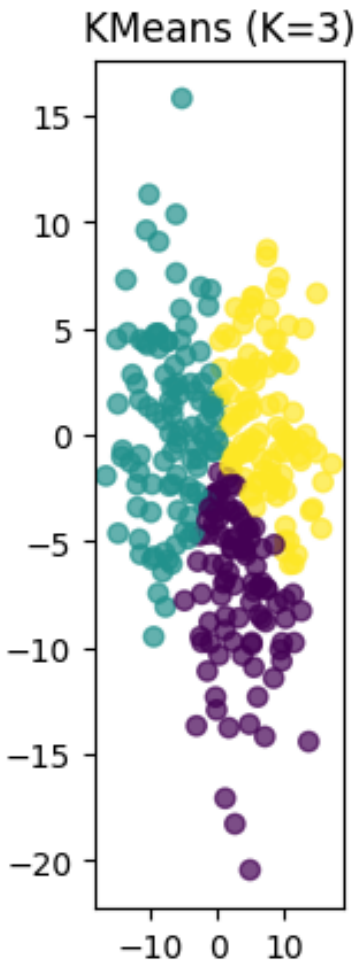
```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
import matplotlib.pyplot as plt

X, label = make_blobs(
    n_samples=300,
    n_features=2,
    centers=3,
    cluster_std=5,
    random_state=123
)

plt.figure(figsize=(15,4))

plt.subplot(1,4,1)
plt.scatter(X[:,0], X[:,1], c=label, alpha=0.7)
plt.title("Original Data")
```

```
Text(0.5, 1.0, 'Original Data')
```



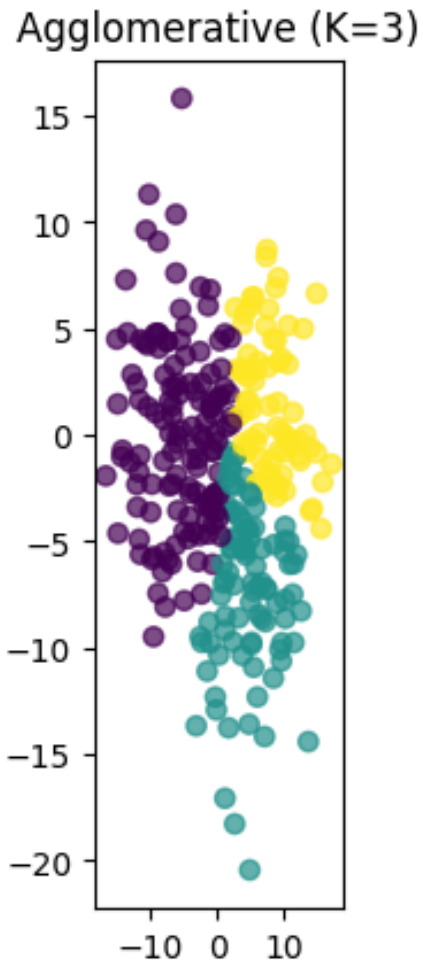

```
#KMeans
kmeans = KMeans(n_clusters=3, random_state=123)
k_labels = kmeans.fit_predict(X)

plt.subplot(1,4,2)
plt.scatter(X[:,0], X[:,1], c=k_labels, alpha=0.7)
plt.title("KMeans (K=3)")
```

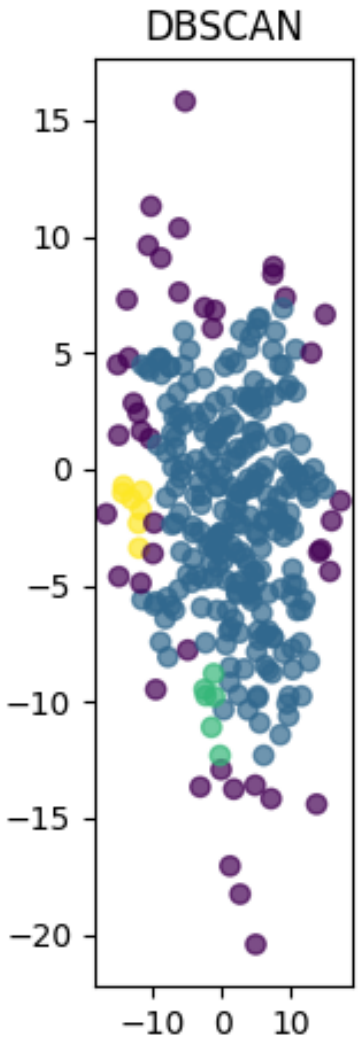
```
Text(0.5, 1.0, 'KMeans (K=3)')
```




```
#Agglomerative
agg = AgglomerativeClustering(n_clusters=3)
a_labels = agg.fit_predict(X)
plt.subplot(1,4,3)
plt.scatter(X[:,0], X[:,1], c=a_labels, alpha=0.7)
plt.title("Agglomerative (K=3)")
```

```
Text(0.5, 1.0, 'Agglomerative (K=3)')
```




```
#DBSCAN
dbscan = DBSCAN(eps=2, min_samples=5)
d_labels = dbscan.fit_predict(X)

plt.subplot(1,4,4)
plt.scatter(X[:,0], X[:,1], c=d_labels, alpha=0.7)
plt.title("DBSCAN")
```

```
plt.tight_layout()
plt.show()
```